Credit Lectures 4 and 5: Feed-Forward Neural Networks

Deep Learning for Actuarial Modelling: a credit risk companion

AUTHOR

Mario Ervedosa, adapting Scognamiglio, Maggi and Wüthrich

PUBLISHED

August 31, 2026

ABSTRACT

We move from the logistic GLM of the second credit lecture to its first deep learning successor. The feed-forward neural network is presented as in Lectures 4 and 5 of Deep Learning for Actuarial Modeling: a learned feature extractor composed with a GLM readout, trained by stochastic gradient descent with early stopping. A network is fitted to the Bondora modelling table in PyTorch, compared against the GLM benchmark on both the random and the out-of-time split, and the balance property of the canonical-link GLM is shown to be the first casualty of leaving it.

Introduction

The second credit lecture closed with a promise. The GLM regression function is linear in the covariates up to the link, and the networks to come would keep every other piece of the machinery (the Bernoulli deviance as loss, the sigmoid as output activation, the out-of-sample loss as arbiter) while replacing the covariates $\boldsymbol{X}$ with learned features $\boldsymbol{z}^{(d:1)}(\boldsymbol{X})$. The third lecture sketched how that substitution works and where it leads. This lecture keeps the promise in full, adapting Lectures 4 and 5 of the course, on feed-forward neural networks, to the Bondora PD problem.

The plan mirrors the course. We first define the feed-forward network architecture and its parameter count, then state the universality theorems that justify the architecture, then

work through gradient descent training, whose critical ingredient is early stopping. The example section fits a network to the Bondora modelling table of the second lecture and asks the questions that lecture taught us to ask, i.e. how the network compares with the GLM on the random split, whether its advantage survives the out-of-time split, and what happens to calibration in the large once the canonical-link score equations are gone.

1 Feed-forward neural networks

1.1 Feature extractor and GLM readout

A feed-forward neural network (FNN) generalises the GLM in one specific place. The GLM regression function of the second lecture reads

$$\boldsymbol{X} \mapsto g\left(\mu_{\vartheta}(\boldsymbol{X})\right) = \langle \vartheta, \boldsymbol{X} \rangle,$$

and its weakness is that the modeller must supply the right covariates by hand: the age bands, the interactions, the transformations. The FNN keeps the GLM structure as its final layer, the **readout**, and prepends a **feature extractor** $\boldsymbol{z}^{(d:1)} : \mathbb{R}^{q_0} \to \mathbb{R}^{q_d}$ that is itself learned from the data,

$$\boldsymbol{X} \mapsto \mu(\boldsymbol{X}) = g^{-1}\left\langle \boldsymbol{w}^{(d+1)}, \boldsymbol{z}^{(d:1)}(\boldsymbol{X}) \right\rangle .$$

For our Bernoulli responses the inverse link g^{-1} is the sigmoid, so the readout is exactly the logistic GLM of the second lecture, applied to learned features in place of raw covariates. A scorecard reader can take this as the definition of a neural network PD model: a logistic regression whose covariates were engineered by the optimiser rather than by the analyst.

1.2 Network layers and the feature extractor

The feature extractor is a composition of d **FNN layers**. Layer m is a map

$$\boldsymbol{z}^{(m)} : \mathbb{R}^{q_{m-1}} \to \mathbb{R}^{q_m}, \qquad \boldsymbol{x} \mapsto \boldsymbol{z}^{(m)}(\boldsymbol{x}) = \left(\phi\langle \boldsymbol{w}_1^{(m)}, \boldsymbol{x}\rangle, \dots, \phi\langle \boldsymbol{w}_{q_m}^{(m)}, \boldsymbol{x}\rangle\right)^{\top},$$

whose j-th component, the j-th **neuron**, applies a non-linear **activation function** ϕ to an affine function $\langle \boldsymbol{w}_j^{(m)}, \boldsymbol{x}\rangle = w_{j,0}^{(m)} + \sum_{k=1}^{q_{m-1}} w_{j,k}^{(m)} x_k$ of the previous layer's output. The full feature extractor of **depth** d is the composition

$$\boldsymbol{z}^{(d:1)} := \boldsymbol{z}^{(d)} \circ \cdots \circ \boldsymbol{z}^{(1)} : \mathbb{R}^{q_0} \to \mathbb{R}^{q_d},$$

with $q_0 = q$ the number of (pre-processed) covariates and $q_1, \dots, q_d$ the numbers of neurons in the hidden layers. Every neuron is therefore a small GLM whose output feeds the GLMs of the next layer, and the network is a hierarchy of GLMs terminating in the readout.

1.3 Activation functions

The activation ϕ supplies the non-linearity; with a linear ϕ the composition would collapse back to one linear map and the network would be an overparametrised GLM. The course table of common choices carries over unchanged.

activation	$\phi(x)$	**derivative** $\phi'(x)$
sigmoid (logistic)	$\sigma(x) = (1 + e^{-x})^{-1}$	$\phi(x)(1 - \phi(x))$
hyperbolic tangent	$\tanh(x) = 2\sigma(2x) - 1$	$1 - \phi(x)^2$
rectified linear unit (ReLU)	$x\,\mathbb{1}_{x \geq 0}$	$\mathbb{1}_{x > 0}$
sigmoid linear unit (SiLU)	$x\,\sigma(x)$	$\sigma(x)(1 - \phi(x)) + \phi(x)$
Gaussian error linear unit (GELU)	$x\,\Phi(x)$	$\Phi(x) + x\,\Phi'(x)$

In this lecture the sigmoid does double duty. As a candidate hidden activation it is one row of this table, and as the inverse canonical link of the Bernoulli member it is the fixed output activation of every PD network we build. The two roles are independent; following the course examples, our hidden layers use the hyperbolic tangent, and the sigmoid appears only in the readout.

1.4 Dimension of the network parameter

Collecting all weights into one parameter $\vartheta = (\boldsymbol{w}_1^{(1)}, \dots, \boldsymbol{w}^{(d+1)}) \in \mathbb{R}^r$, each layer contributes q_m neurons with $q_{m-1} + 1$ weights each, and the readout contributes $q_d + 1$ more,

$$r = \sum_{m=1}^{d} q_m \left(q_{m-1} + 1\right) + \left(q_d + 1\right).$$

The course illustrates the formula on a small architecture with $q_0 = 16$ covariates and two hidden layers of $q_1 = q_2 = 8$ neurons, giving $r = 8 \cdot 17 + 8 \cdot 9 + 9 = 217$. The same architecture in PyTorch, with the sigmoid readout of a PD model:

```
import numpy as np
import pandas as pd
import polars as pl
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import torch
from sklearn.metrics import roc_auc_score

def make_fnn(q0, widths, seed):
    """FNN of depth len(widths) with tanh activations and sigmoid
        readout."""
    torch.manual_seed(seed)
    layers, q_prev = [], q0
    for q_m in widths:
        layers += [torch.nn.Linear(q_prev, q_m), torch.nn.Tanh()]
        q_prev = q_m
    layers += [torch.nn.Linear(q_prev, 1), torch.nn.Sigmoid()]
    return torch.nn.Sequential(*layers)

toy = make_fnn(q0=16, widths=(8, 8), seed=7)
print(toy)
print("trainable parameters:", sum(p.numel() for p in toy.parameters()))
```

```
Sequential(
  (0): Linear(in_features=16, out_features=8, bias=True)
  (1): Tanh()
  (2): Linear(in_features=8, out_features=8, bias=True)
  (3): Tanh()
  (4): Linear(in_features=8, out_features=1, bias=True)
  (5): Sigmoid()
)
trainable parameters: 217
```

The count agrees with the formula. Two structural facts follow from the architecture and shape everything below. First, μ_ϑ is no longer linear in ϑ in any reparametrisation, so the likelihood is non-concave and nothing guarantees a unique maximiser. Second, the network is heavily overparametrised relative to a GLM, so fitting it to the in-sample optimum would memorise noise. Both facts are addressed by the training procedure of the gradient descent section rather than by the architecture itself.

2 Universality theorems

The justification for the architecture is that it can represent essentially anything. The classical universality theorems (Cybenko, 1989; Hornik, Stinchcombe and White, 1989; Hornik, 1991) state that a network with a single hidden layer and a sigmoidal activation can approximate any continuous function on a compact set to any prescribed accuracy, provided the hidden layer is wide enough. Leshno et al. (1993) sharpened the requirement on the activation: universality holds precisely when ϕ is non-polynomial. Depth is therefore never needed for expressiveness in principle; in practice deep and narrow networks reach a given accuracy with far fewer parameters than shallow and wide ones, which is why the working architectures of this course have two to four hidden layers.

For credit modelling the theorems cut both ways. They promise that the true PD functional $\boldsymbol{X} \mapsto \mathrm{PD}(\boldsymbol{X})$, whatever its shape, lies within reach of the architecture, including every interaction the second lecture had to add by hand. Nevertheless, they promise this on the approximation side only. On finite samples an expressive enough network can also fit the noise perfectly, so the theorems make regularisation a necessity rather than an optional refinement. Moreover, the approximating network is far from unique, so two training runs will land on different parameters of similar quality. Both consequences are visible in the example section.

3 Gradient descent algorithm

3.1 Objective function

Training minimises the in-sample Bernoulli deviance of the second lecture over the network parameter. With unit weights and dispersion ($v_i \equiv 1$, $\varphi = 1$), the objective on a sample $\mathcal{S}$ of size n is

$$\mathcal{L}(\vartheta; \mathcal{S}) = \frac{1}{n} \sum_{i \in \mathcal{S}} 2 \left(-D_i \log \mu_\vartheta(\boldsymbol{X}_i) - (1 - D_i) \log \left(1 - \mu_\vartheta(\boldsymbol{X}_i)\right) \right),$$

twice the binary cross-entropy that every deep learning library ships as `BCELoss`. For the GLM this objective was smooth and convex and IRLS solved it exactly. For the FNN it is non-convex, and the workhorse is **gradient descent**: from a random initial value $\vartheta^{[0]}$, iterate

$$\vartheta^{[t]} \;\mapsto\; \vartheta^{[t+1]} = \vartheta^{[t]} - \rho_{t+1} \nabla_\vartheta \, \mathcal{L}\left(\vartheta^{[t]}; \mathcal{S}\right),$$

with **learning rates** $\rho_t > 0$, taking a small step against the gradient at every iteration. The gradient of the composition is computed layer by layer with the chain rule, the **back-propagation** algorithm of Rumelhart, Hinton and Williams (1986), and automatic differentiation frameworks such as PyTorch make its implementation invisible to the modeller.

3.2 Covariate pre-processing

Gradient descent imposes a requirement that IRLS never did. The GLM estimate is equivariant to affine rescaling of the covariates (rescaling a covariate rescales its coefficient and changes nothing else), so the second lecture could feed raw ages and log-incomes without a thought. Gradient descent shares one learning rate across all components of ϑ, and a covariate living on $[1{,}000, 10{,}000]$ next to one living on $[0, 1]$ produces gradients of hopelessly different sizes on the two coordinates. The course therefore pre-processes every covariate, and we adopt its pipeline:

- continuous covariates are **standardised** to mean zero and unit variance, $X \mapsto (X - \hat{m})/\hat{s}$, with $\hat{m}$ and $\hat{s}$ estimated on the training data;
- rare tails of continuous covariates are **censored** before standardisation, because a handful of extreme values otherwise dominates the scale;
- categorical covariates are **one-hot encoded**, one indicator column per level. The redundancy of the full encoding (the columns sum to one) matters for the GLM's design matrix rank; the network's non-convex fit carries no such rank condition, so the full encoding is standard.

Entity embeddings, the network-native treatment of high-cardinality categoricals, are the subject of the next lecture; our categoricals have at most twelve levels, so one-hot encoding is adequate here.

3.3 Early stopping: the most critical part

The course flags early stopping as the most important element of network training, and the reason is the overparametrisation noted above. Run to convergence, gradient descent drives the in-sample deviance towards its minimum and the network towards memorising the learning sample. The remedy is to track generalisation during training and stop when it stops improving. Concretely, the learning sample $\mathcal{L}$ is partitioned into a **training sample** $\mathcal{U}$ and a **validation sample** $\mathcal{V}$; gradient steps use $\mathcal{U}$ only, while $\mathcal{L}(\vartheta^{[t]}; \mathcal{V})$ is evaluated after every epoch. Training deviance falls monotonically, validation deviance falls and then turns upward, and the parameter retained is the one with the smallest validation deviance ever seen, a scheme called checkpointing. The **test sample** $\mathcal{T}$ takes no part in any of this and is touched once, for the final model comparison. Early stopping is thus a form of implicit regularisation, limiting the network's capacity by how far from initialisation the parameter is allowed to travel rather than by an explicit penalty term. Explicit alternatives exist, above all **dropout** (Srivastava et al., 2014), which disables a random subset of neurons at every gradient step, and classical ridge or LASSO penalties on the weights; the course, and this lecture, get by with early stopping alone.

3.4 Stochastic gradient descent and optimisers

Two refinements make gradient descent practical at portfolio scale. First, computing the exact gradient means a pass over the full training sample per step, which is wasteful when a small random subset already points in roughly the right direction. **Stochastic gradient descent** (SGD) therefore partitions each **epoch** into random **mini-batches** and takes one gradient step per batch. The steps are noisier, and the noise is useful, since it lets the iteration escape shallow local optima of the non-convex objective. Second, the plain update above is sensitive to the learning-rate schedule. Momentum-based optimisers smooth the trajectory by accumulating past gradients (Nesterov, 2007), and the adaptive family around **Adam** (Kingma and Ba, 2017) additionally maintains a per-coordinate

learning rate. We use NAdam, the Nesterov-accelerated variant that the course's Keras examples call `nadam`, with its default learning rate.

4 FNN example: Bondora PD data

4.1 Modelling table and design matrix

We reuse the modelling frame of the second lecture without change: loans with a recorded age and positive income, the unknown education code dropped, log-income as the income scale, and explicit `missing` categories for home ownership and employment duration.

```
dat = pl.read_parquet("../data/bondora_pd.parquet")
model_dat = (
    dat.filter(pl.col("Age").is_not_null() & (pl.col("IncomeTotal") > 0))
    .with_columns(logIncome=pl.col("IncomeTotal").log())
    .filter(pl.col("Education") != -1)
)
g = model_dat.select(
    ["default_12m", "Age", "logIncome", "Country", "Education",
     "HomeOwnershipType", "NewCreditCustomer", "VerificationType",
     "EmploymentDurationCurrentEmployer", "LoanDate"]).to_pandas()

g["AgeClass"] = pd.cut(
    g["Age"], [18, 20, 25, 30, 40, 50, 60, 101],
    labels=["18-20", "21-25", "26-30", "31-40", "41-50", "51-60", "61+"],
    include_lowest=True)
ho = g["HomeOwnershipType"]
g["HomeOwnership"] = np.where(ho.isna() | (ho == -1), "missing",
                              ho.fillna(-1).astype(int).astype(str))
g["EmploymentDuration"] =
        g["EmploymentDurationCurrentEmployer"].fillna("missing")
print(f"modelling sample: n = {len(g):,}")
```

```
modelling sample: n = 148,667
```

The GLM benchmark keeps the second lecture's banded design, with `AgeClass` cut at the seven hand-chosen boundaries. The network receives the same information in the course's pre-processed form instead: age and log-income as standardised continuous covariates, and the five categoricals one-hot encoded. The contrast is deliberate and mirrors the course's own comparison, where the GLM works on hand-banded covariates and the network is left to learn the shapes itself. Following the course's censoring of rare tails (driver age at 90, bonus-malus at 150), we censor age at 70; only 13 loans are older, with a maximum of 77, and those few observations should not stretch the standardised scale.

```
g["AgeCens"] = g["Age"].clip(upper=70)
onehot = pd.get_dummies(
    pd.DataFrame({
        "Country": g["Country"],
        "Education": g["Education"].astype(int).astype(str),
        "HomeOwnership": g["HomeOwnership"],
        "VerificationType": g["VerificationType"].astype(int).astype(str),
        "EmploymentDuration": g["EmploymentDuration"],
    }), prefix_sep=":")
X_all = pd.concat([
    g[["AgeCens", "logIncome"]],
    g["NewCreditCustomer"].astype(float),
    onehot.astype(float),
], axis=1)
print(f"design matrix: q0 = {X_all.shape[1]} columns")
```

```
design matrix: q0 = 38 columns
```

The design matrix carries $q_0 = 38$ pre-processed covariates, a close parallel to the course's 40 for the French MTPL data. Standardisation is deferred to the training function below, because $\hat{m}$ and $\hat{s}$ must be estimated on training data only; estimating them on the full sample would leak the test set's location and scale into the fit.

4.2 Splits and benchmark GLMs

The learning and test samples are rebuilt exactly as in the second lecture, with the same seeds: an 80/20 random split, and an out-of-time split at the 0.8 quantile of origination dates.

```
rng = np.random.default_rng(1)
perm = rng.permutation(len(g))
n_learn = int(0.8 * len(g))
idx_learn_r, idx_test_r = perm[:n_learn], perm[n_learn:]
learn_r, test_r = g.iloc[idx_learn_r], g.iloc[idx_test_r]

cutoff = g["LoanDate"].quantile(0.8)
mask_t = (g["LoanDate"] <= cutoff).to_numpy()
learn_t, test_t = g[mask_t], g[~mask_t]
print(f"random: learn {len(learn_r):,} / test {len(test_r):,};   "
      f"out-of-time: learn {len(learn_t):,} / test {len(test_t):,} "
      f"(cutoff {cutoff.date()})")
```

```
random: learn 118,933 / test 29,734;   out-of-time: learn 118,998 / test
29,669 (cutoff 2019-11-04)
```

The two benchmark GLMs refit the second lecture's formula on each learning sample, and the evaluation helpers carry over unchanged, including the null-model deviance that makes cross-sample comparisons honest.

```
FORMULA = ("default_12m ~ AgeClass + logIncome + Country + C(Education)"
           " + C(HomeOwnership) + NewCreditCustomer + C(VerificationType)"
           " + C(EmploymentDuration)")
glm_r = smf.glm(FORMULA, data=learn_r, family=sm.families.Binomial()).fit()
glm_t = smf.glm(FORMULA, data=learn_t, family=sm.families.Binomial()).fit()

def bernoulli_deviance(pred, obs):
    eps = 1e-12
    p = np.clip(np.asarray(pred, float), eps, 1 - eps)
    y = np.asarray(obs, float)
    return 100 * 2 * np.mean(-y * np.log(p) - (1 - y) * np.log(1 - p))

def report(label, pred, obs):
    null = np.full(len(obs), np.asarray(obs, float).mean())
    print(f"{label}: deviance {bernoulli_deviance(pred, obs):7.3f}   "
          f"null {bernoulli_deviance(null, obs):7.3f}   "
          f"AUC {roc_auc_score(obs, pred):.4f}   "
          f"mean PD {np.mean(pred):.4f} vs observed {np.mean(obs):.4f}")
```

4.3 Training the network

The training function implements the full recipe of the gradient descent section: a 90/10 partition of the learning sample into $\mathcal{U}$ and $\mathcal{V}$, standardisation fitted on $\mathcal{U}$, NAdam over mini-batches of 5,000 loans, and early stopping by checkpointing on the validation deviance, abandoning training once fifty epochs pass without improvement. All seeds are fixed and training runs on the CPU, so the numbers below reproduce on re-render; on this machine's Apple silicon `torch.backends.mps.is_available()` is true, and the model is small enough that the GPU would win nothing.

```
def train_fnn(X_learn, y_learn, seed=7, epochs=500, batch=5000,
        patience=50):
    p = np.random.default_rng(seed).permutation(len(X_learn))
    tr, va = p[:int(0.9 * len(p))], p[int(0.9 * len(p)):]
    m_hat, s_hat = X_learn.iloc[tr].mean(), X_learn.iloc[tr].std()
    s_hat = s_hat.replace(0, 1.0)  # a one-hot level can be absent from U

    def prep(df):
        return torch.tensor(((df - m_hat) /
        s_hat).to_numpy(dtype=np.float32))

    Xtr, Xva = prep(X_learn.iloc[tr]), prep(X_learn.iloc[va])
    ytr =
        torch.tensor(y_learn.iloc[tr].to_numpy(dtype=np.float32)).unsqueeze(1)
    yva =
        torch.tensor(y_learn.iloc[va].to_numpy(dtype=np.float32)).unsqueeze(1)

    model = make_fnn(X_learn.shape[1], widths=(20, 15, 10), seed=seed)
    opt = torch.optim.NAdam(model.parameters())
    bce = torch.nn.BCELoss()
    gen = torch.Generator().manual_seed(seed)

    best, best_epoch, best_state, hist = np.inf, 0, None, []
    for epoch in range(1, epochs + 1):
        model.train()
        for chunk in torch.randperm(len(Xtr), generator=gen).split(batch):
            opt.zero_grad()
            loss = bce(model(Xtr[chunk]), ytr[chunk])
            loss.backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            hist.append((200 * bce(model(Xtr), ytr).item(),
                         200 * bce(model(Xva), yva).item()))
        if hist[-1][1] < best:
            best, best_epoch = hist[-1][1], epoch
            best_state = {k: v.clone() for k, v in
        model.state_dict().items()}
        if epoch - best_epoch >= patience:
            break
    model.load_state_dict(best_state)
    return model, (m_hat, s_hat), np.array(hist), best_epoch

def fnn_predict(model, scaler, X):
    m_hat, s_hat = scaler
    with torch.no_grad():
```

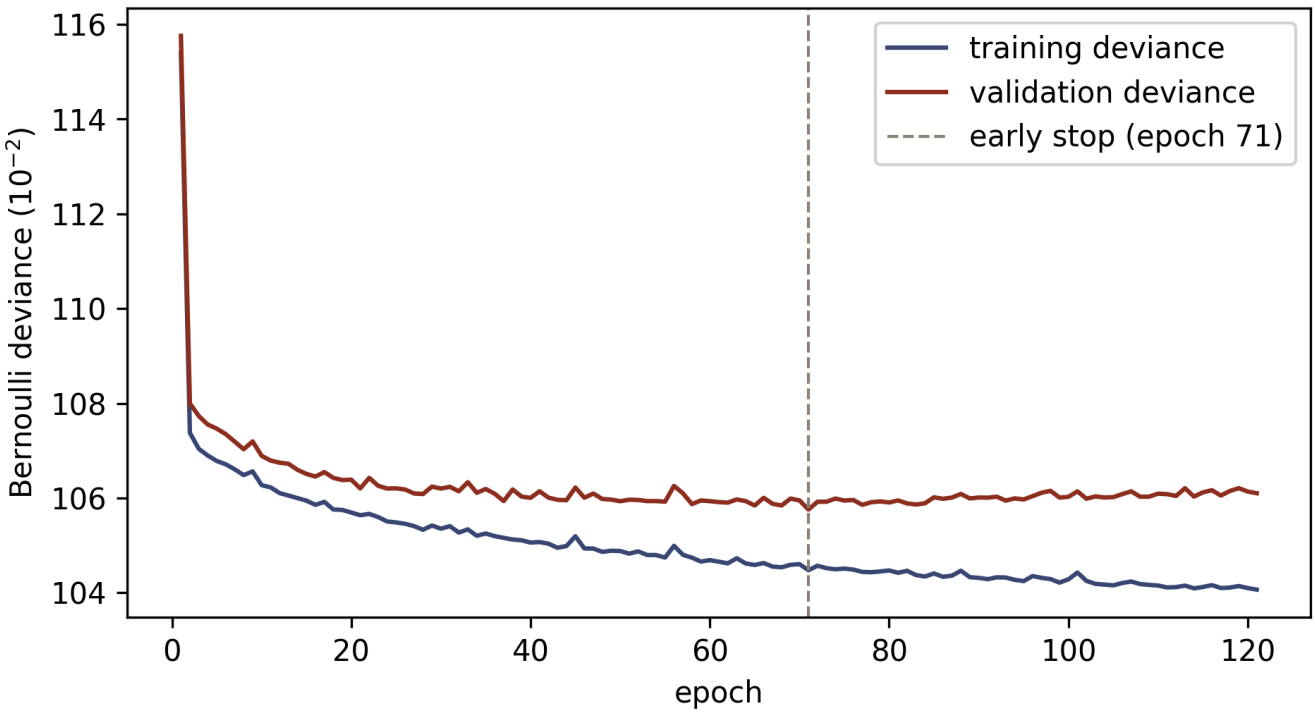
```
        return model(torch.tensor(
            ((X - m_hat) /
        s_hat).to_numpy(dtype=np.float32))).squeeze(1).numpy()
```

We fit the network on the random split's learning sample. The chosen architecture copies the course's French MTPL network, depth $d = 3$ with $(q_1, q_2, q_3) = (20, 15, 10)$ neurons, which on $q_0 = 38$ inputs gives $r = 20 \cdot 39 + 15 \cdot 21 + 10 \cdot 16 + 11 = 1{,}266$ trainable parameters against the course's 1,306.

```
X_learn_r, y_all = X_all.iloc[idx_learn_r], g["default_12m"].astype(float)
fnn_r, sc_r, hist_r, stop_r = train_fnn(X_learn_r, y_all.iloc[idx_learn_r])
print(f"parameters: {sum(p.numel() for p in fnn_r.parameters()):,}; "
      f"validation optimum at epoch {stop_r} of {len(hist_r)} run")
```

```
parameters: 1,266; validation optimum at epoch 71 of 121 run
```

```
fig, ax = plt.subplots(figsize=(6.5, 3.6))
epochs_axis = np.arange(1, len(hist_r) + 1)
ax.plot(epochs_axis, hist_r[:, 0], color="#3B4A75", label="training
        deviance")
ax.plot(epochs_axis, hist_r[:, 1], color="#8C2F1E", label="validation
        deviance")
ax.axvline(stop_r, color="#8A8377", ls="--", lw=1, label=f"early stop
        (epoch {stop_r})")
ax.set_xlabel("epoch"); ax.set_ylabel(r"Bernoulli deviance ($10^{-2}$)")
ax.legend(); plt.tight_layout(); plt.show()
```



Training and validation Bernoulli deviance by epoch on the random split, our computed counterpart of the course's early-stopping illustration. The training deviance keeps falling; the validation deviance bottoms out at epoch 71 (dashed line) and then drifts upward as the network starts memorising the training sample. The retained parameter is the epoch-71 checkpoint.

4.4 Results on the random split

```
report("GLM, in-sample ", glm_r.fittedvalues, learn_r["default_12m"])
report("GLM, test      ", glm_r.predict(test_r), test_r["default_12m"])
report("FNN, in-sample ", fnn_predict(fnn_r, sc_r, X_learn_r),
        y_all.iloc[idx_learn_r])
report("FNN, test      ", fnn_predict(fnn_r, sc_r, X_all.iloc[idx_test_r]),
       y_all.iloc[idx_test_r])
```

```
GLM, in-sample : deviance 106.641   null 120.210   AUC 0.7263   mean PD
0.2888 vs observed 0.2888
GLM, test      : deviance 108.197   null 120.802   AUC 0.7182   mean PD
0.2888 vs observed 0.2921
FNN, in-sample : deviance 104.606   null 120.210   AUC 0.7416   mean PD
0.2908 vs observed 0.2888
FNN, test      : deviance 107.621   null 120.802   AUC 0.7241   mean PD
0.2911 vs observed 0.2921
```

On the random split the network earns its keep. Its out-of-sample deviance of 107.621 improves on the GLM's 108.197, and its test AUC of 0.724 improves on the GLM's 0.718, so the learned features find structure that the hand-banded design missed. The in-sample gap tells the same story from the other side. The FNN sits 3.0 deviance points below its own test value where the GLM sits only 1.6 below, which is the extra flexibility showing, held in check by early stopping. The gains are modest in absolute terms, and the course's MTPL example behaves identically. A well-built GLM is a strong benchmark, and the network's advantage is real, incremental and bought with 1,266 parameters in place of 39.

Non-uniqueness deserves its promised demonstration. Retraining the same architecture on the same data from a different random initialisation lands on different weights and, in general, a slightly different out-of-sample performance.

```
fnn_r2, sc_r2, hist_r2, stop_r2 = train_fnn(X_learn_r,
        y_all.iloc[idx_learn_r],
                                           seed=17)
report("FNN, test (seed 17)", fnn_predict(fnn_r2, sc_r2,
        X_all.iloc[idx_test_r]),
       y_all.iloc[idx_test_r])
```

```
FNN, test (seed 17): deviance 107.475   null 120.802   AUC 0.7249   mean PD
0.2905 vs observed 0.2921
```

The second run stops at epoch 83 rather than 71 and lands within 0.15 deviance points of the first, on the favourable side this time. The ordering relative to the GLM is stable across seeds, and the residual run-to-run variation is exactly the estimation noise that the ensembling techniques of the next lecture average away.

4.5 The balance property is the first casualty

The second lecture proved that the canonical-link GLM's fitted PDs sum to the observed defaults on the learning sample, and called it calibration in the large. That proof ran through the maximum likelihood score equations, which the network no longer solves. Gradient descent stops early, deliberately short of any likelihood optimum.

```
fit_fnn = fnn_predict(fnn_r, sc_r, X_learn_r)
print(f"GLM: sum of fitted PDs {glm_r.fittedvalues.sum():,.1f}   "
      f"FNN: sum of fitted PDs {fit_fnn.sum():,.1f}   "
      f"observed defaults {int(y_all.iloc[idx_learn_r].sum()):,}")
```

```
GLM: sum of fitted PDs 34,345.0   FNN: sum of fitted PDs 34,582.2
observed defaults 34,345
```

The GLM reproduces the 34,345 observed defaults to machine precision. The network overshoots them by 237, roughly 0.7 per cent of the portfolio's defaults, despite having the better deviance. For insurance pricing the analogous failure means collecting the wrong total premium; for a credit portfolio it means a mis-stated expected loss at the aggregate level, which is the level capital planning cares about. The course addresses this

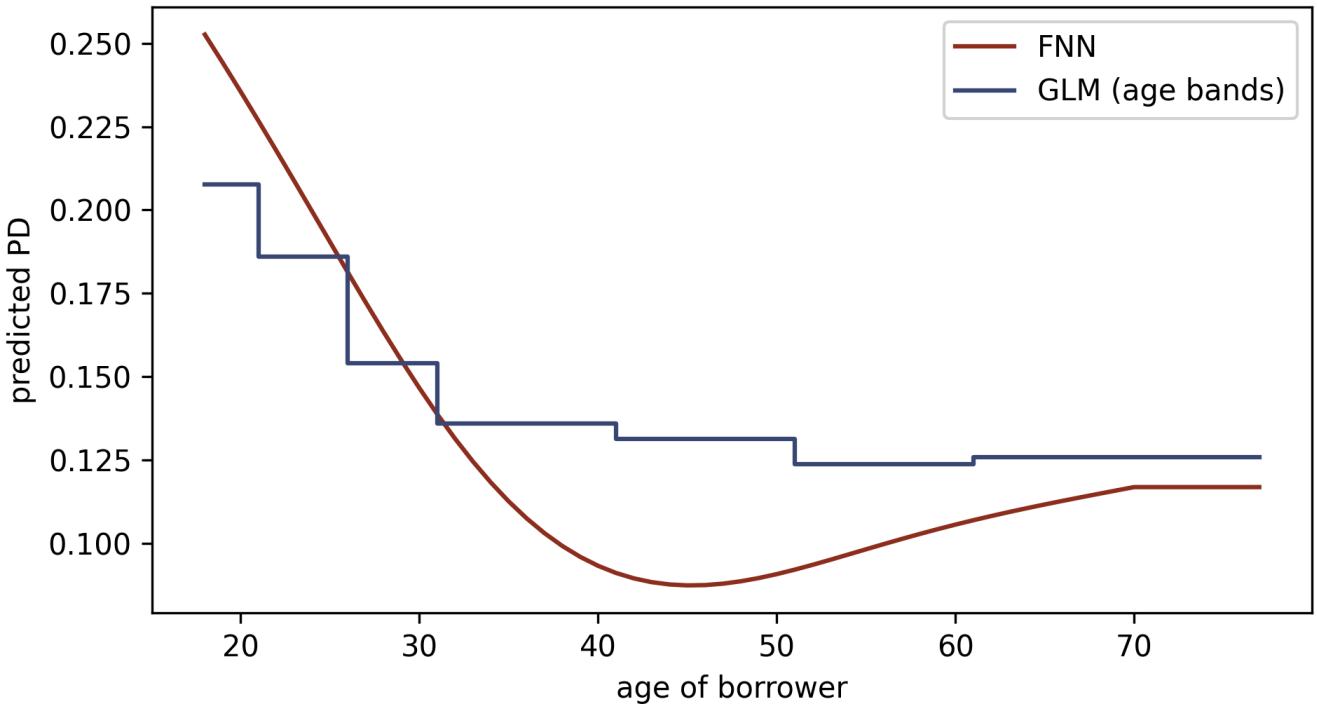
with bias regularisation and balance corrections, and the credit series takes it up in the calibration companion; the point to hold on to is that leaving the canonical-link GLM silently forfeits an exactness guarantee that scorecard practice has always taken for granted.

4.6 The learned age effect

The second lecture's age bands were hand-crafted, with boundaries chosen by judgement. The network was handed age as a single standardised column and left to find the shape itself. We can inspect what it found by sweeping age across its range for a fixed reference borrower (the learning sample's modal profile: an Estonian new customer, education level 4, home ownership type 1, income verified to level 4, employed more than five years, at the median log-income) and plotting the predicted PD against the GLM's step function for the same profile.

```
modal = {c: learn_r[c].mode()[0] for c in
         ["Country", "Education", "HomeOwnership", "VerificationType",
          "EmploymentDuration", "NewCreditCustomer"]}
sweep = pd.DataFrame({"Age": np.arange(18, 78),
                      "logIncome": learn_r["logIncome"].median(), **modal})
sweep["AgeCens"] = sweep["Age"].clip(upper=70)
sweep["AgeClass"] = pd.cut(
    sweep["Age"], [18, 20, 25, 30, 40, 50, 60, 101],
    labels=["18-20", "21-25", "26-30", "31-40", "41-50", "51-60", "61+"],
    include_lowest=True)
sweep_oh = pd.get_dummies(
    pd.DataFrame({
        "Country": sweep["Country"],
        "Education": sweep["Education"].astype(int).astype(str),
        "HomeOwnership": sweep["HomeOwnership"],
        "VerificationType":
         sweep["VerificationType"].astype(int).astype(str),
        "EmploymentDuration": sweep["EmploymentDuration"],
    }), prefix_sep=":").astype(float)
sweep_X = pd.concat([sweep[["AgeCens", "logIncome"]],
                     sweep["NewCreditCustomer"].astype(float), sweep_oh],
                    axis=1).reindex(columns=X_all.columns, fill_value=0.0)

fig, ax = plt.subplots(figsize=(6.5, 3.6))
ax.plot(sweep["Age"], fnn_predict(fnn_r, sc_r, sweep_X),
        color="#8C2F1E", label="FNN")
ax.plot(sweep["Age"], glm_r.predict(sweep), color="#3B4A75",
        drawstyle="steps-post", label="GLM (age bands)")
ax.set_xlabel("age of borrower"); ax.set_ylabel("predicted PD")
ax.legend(); plt.tight_layout(); plt.show()
```



Predicted PD against borrower age for the reference profile, network against GLM, fitted on the random split's learning sample. The GLM is constant within each hand-chosen age band; the network traces a smooth curve through the same territory, steep to age 40, flat at the bottom through the 40s and 50s, and rising gently into old age. Above the censoring point of 70 the network's input is constant, so its curve is flat by construction.

The two models agree on the qualitative story, a strong decline in PD from 18 to about 40 followed by a shallow trough and a mild rise for the oldest borrowers. They disagree instructively on the details. The network starts higher for the youngest borrowers (25.3 per cent at age 18 against the GLM's 20.8) and dips lower at the trough (9.3 per cent around age 40 against 13.6), so the banded GLM was averaging away real variation within its widest bands. This figure is the feature extractor made visible. Without being told where the boundaries were, $z^{(d:1)}$ has rebuilt the age banding as a smooth function.

4.7 Does the advantage survive the out-of-time split?

The second lecture's out-of-time exercise found the GLM's ranking intact and its level about eight percentage points too high on the pandemic-era test window. The network now takes the same examination: fitted on the out-of-time learning window, evaluated on the held-out final vintages.

```
X_learn_t = X_all[mask_t]
fnn_t, sc_t, hist_t, stop_t = train_fnn(X_learn_t, y_all[mask_t])
print(f"validation optimum at epoch {stop_t} of {len(hist_t)} run")
report("GLM, in-sample ", glm_t.fittedvalues, learn_t["default_12m"])
report("GLM, oot test  ", glm_t.predict(test_t), test_t["default_12m"])
report("FNN, in-sample ", fnn_predict(fnn_t, sc_t, X_learn_t),
       y_all[mask_t])
report("FNN, oot test  ", fnn_predict(fnn_t, sc_t, X_all[~mask_t]),
       y_all[~mask_t])
```

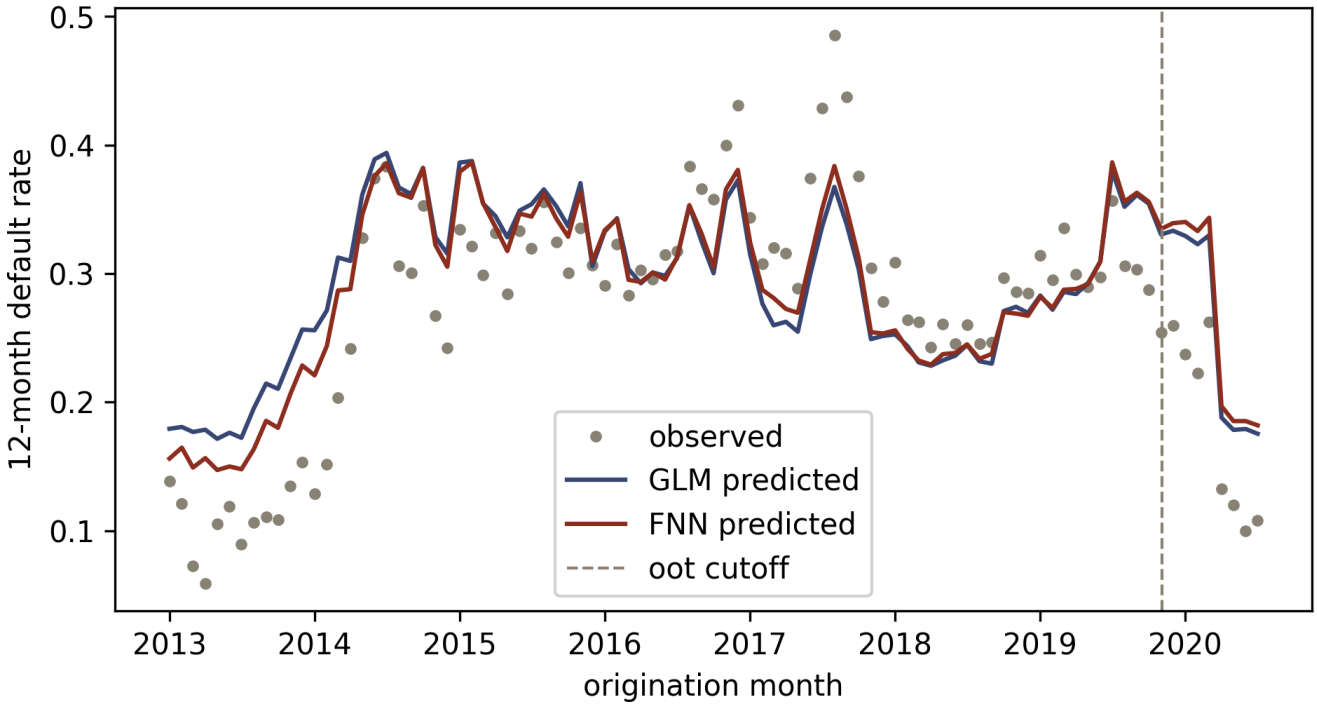
```
validation optimum at epoch 154 of 204 run
GLM, in-sample : deviance 108.873   null 122.969   AUC 0.7274   mean PD
0.3048 vs observed 0.3048
GLM, oot test  : deviance 100.400   null 107.365   AUC 0.7158   mean PD
0.3072 vs observed 0.2280
FNN, in-sample : deviance 106.014   null 122.969   AUC 0.7481   mean PD
0.3057 vs observed 0.3048
FNN, oot test  : deviance 101.329   null 107.365   AUC 0.7146   mean PD
0.3156 vs observed 0.2280
```

The answer is no, and the numbers are worth reading slowly. In-sample, the network repeats its random-split advantage, 106.014 against the GLM's 108.873. Out of time, the advantage reverses: the network's test deviance of 101.329 sits above the GLM's 100.400, its AUC of 0.715 sits marginally below the GLM's 0.716, and its mean predicted PD of 31.6 per cent overshoots the observed 22.8 per cent by nearly nine percentage points, where the GLM predicted 30.7. Every quality that made the network better on exchangeable data made it at best equal on drifted data, because the structure it so faithfully extracted from the development window is precisely what the pandemic-era book departed from.

For credit modelling, the vintage drift of this portfolio matters more than the choice of model class. Moving from 39 GLM parameters to 1,266 network parameters shifts the out-of-time deviance by under one point, while the drift itself costs both models an eight-to-nine point calibration miss. Consequently, a modeller with an hour to spend on this book should spend it on the stability problem before the flexibility problem. The monthly picture makes the diagnosis visible.

```
g2 = g.copy()
g2["GLM"] = glm_t.predict(g2)
g2["FNN"] = fnn_predict(fnn_t, sc_t, X_all)
g2["month"] = g2["LoanDate"].dt.to_period("M").dt.to_timestamp()
monthly = (g2[g2["month"] >= "2013-01-01"]
           .groupby("month")[["default_12m", "GLM", "FNN"]].mean())

fig, ax = plt.subplots(figsize=(6.5, 3.6))
ax.plot(monthly.index, monthly["default_12m"], "o", ms=3,
        color="#8A8377", label="observed")
ax.plot(monthly.index, monthly["GLM"], color="#3B4A75", label="GLM
        predicted")
ax.plot(monthly.index, monthly["FNN"], color="#8C2F1E", label="FNN
        predicted")
ax.axvline(cutoff, color="#8A8377", ls="--", lw=1, label="oot cutoff")
ax.set_xlabel("origination month"); ax.set_ylabel("12-month default rate")
ax.legend(); plt.tight_layout(); plt.show()
```



Monthly observed default rate (dots) against the mean predicted PD of the out-of-time GLM and FNN. The two models are nearly indistinguishable at monthly resolution, inside and outside the development window alike; right of the dashed cutoff both track the vintage cycle they were fitted on while the observed rates fall away beneath them. Months from 2013 onward are shown, as in the second lecture.

Takeaways and outlook

A feed-forward network is a parametrised class of regression functions, fixed by the depth d, the neuron counts $q_1, \dots, q_d$, the activation ϕ and the output activation g^{-1}, and trained by stochastic gradient descent on the same Bernoulli deviance the GLM minimised. The universality theorems guarantee the class is rich enough for any continuous PD functional; the price of that richness is a non-convex fit with no unique optimum, held away from memorisation by early stopping on a validation sample.

On the Bondora book the network delivered what the theory suggests. It beat the GLM out-of-sample on exchangeable data, it recovered the hand-crafted age structure as a smooth learned feature, and it gave up the balance property, missing the observed default count by 0.7 per cent where the canonical-link GLM was exact. Out of time, the vintage drift swamped the model-class difference entirely, and the network's calibration miss exceeded the GLM's. Hence the deep learning agenda for credit turns on the surrounding discipline of calibration repair, stability across vintages, and honest uncertainty about a non-unique fit, at least as much as on raw accuracy.

The next lecture takes up two of those threads directly. Ensembling (nagging) averages away the run-to-run randomness the seed experiment exposed, and entity embeddings replace one-hot encoding for categorical covariates, learning a low-dimensional representation per level. Both are mechanical extensions of the architecture built here.

Copyright and attribution

learned age-effect figure and the monthly drift figure are new, and the numerical results concern the credit data throughout. The original lecture notes are available at https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5162304.

References

- Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems* 2(4), 303-314.
- Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks* 4(2), 251-257.
- Hornik, K., Stinchcombe, M. and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks* 2(5), 359-366.
- Kingma, D.P. and Ba, J. (2017). Adam: a method for stochastic optimization. arXiv:1412.6980.
- Leshno, M., Lin, V.Y., Pinkus, A. and Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. *Neural Networks* 6(6), 861-867.
- Nesterov, Y. (2007). Gradient methods for minimizing composite objective function. CORE Discussion Paper 2007/76.
- Rumelhart, D.E., Hinton, G.E. and Williams, R.J. (1986). Learning representations by back-propagating errors. *Nature* 323, 533-536.
- Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. and Salakhutdinov, R. (2014). Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research* 15(56), 1929-1958.
- Wüthrich, M.V. and Merz, M. (2023). *Statistical Foundations of Actuarial Learning and its Applications*. Springer.
- Scognamiglio, S., Maggi, M. and Wüthrich, M.V. (2026). *Deep Learning for Actuarial Modeling*, SSRN 5162304.